Web Application Penetration Testing on OWASP Juice Shop using Burp Suite Report

---

Prepared By:- Sivakumar Nadikattu

Date:- 24/04/2026

Introduction

The objective of this penetration testing project was to identify and test security vulnerabilities in the OWASP Juice Shop web application using the OWASP Top 10 framework. The purpose of this assessment was to evaluate the security posture of the application and identify common web vulnerabilities that may lead to unauthorized access, authentication bypass, improper input validation, and security misconfiguration. The testing was performed using Burp Suite Community Edition in a controlled lab environment using Kali Linux.

Methodology

The assessment was conducted using a black-box penetration testing approach, where the application was tested from an attacker's perspective without internal system access.

Approach Used

- Reconnaissance using Burp Suite
- Proxy interception of HTTP requests
- HTTP History analysis
- Manual request modification using Repeater
- Payload injection for vulnerability testing
- Response analysis
- Repeatable testing for validation
- Mapping vulnerabilities to OWASP Top 10 categories

This approach helped identify multiple security weaknesses in the application.

Tools Used

The following tools were used during testing:

- VMware Workstation
- Kali Linux

- Docker
- Burp Suite Community Edition
- Mozilla Firefox
- Google Chrome
- OWASP Juice Shop

These tools helped in intercepting, modifying, and analyzing application requests and responses.

Procedure

Step 1 – Open VMware Workstation and Start Kali Linux

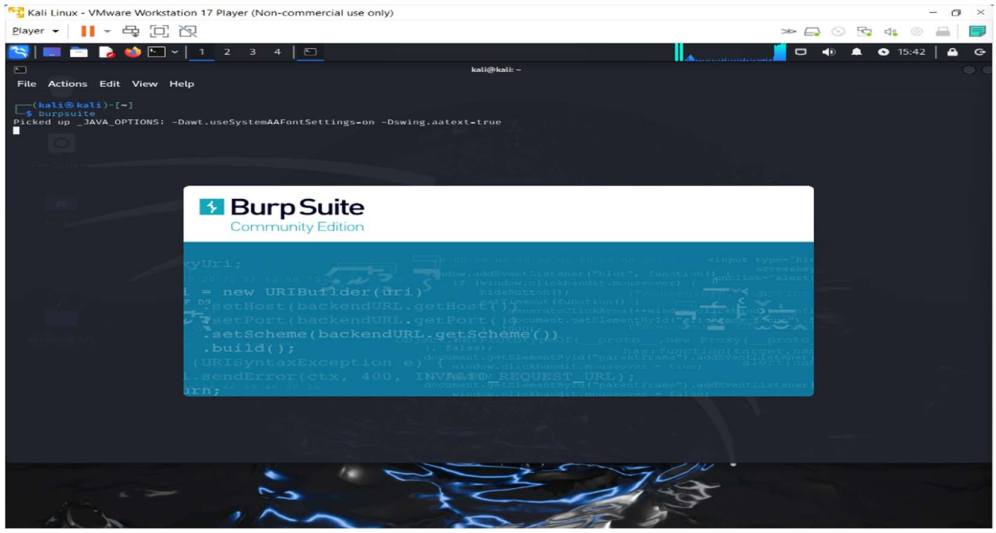
First, VMware Workstation was opened on the host system and the Kali Linux virtual machine was started. Kali Linux was used as the penetration testing environment because it contains pre-installed security testing tools such as Burp Suite and browser support for web application testing.

Step 2 – Launch Burp Suite

After Kali Linux loaded successfully, the Terminal was opened and the following command was executed:

Burpsuite

This launched Burp Suite Community Edition, which was used for intercepting, analyzing, and modifying HTTP requests during testing.



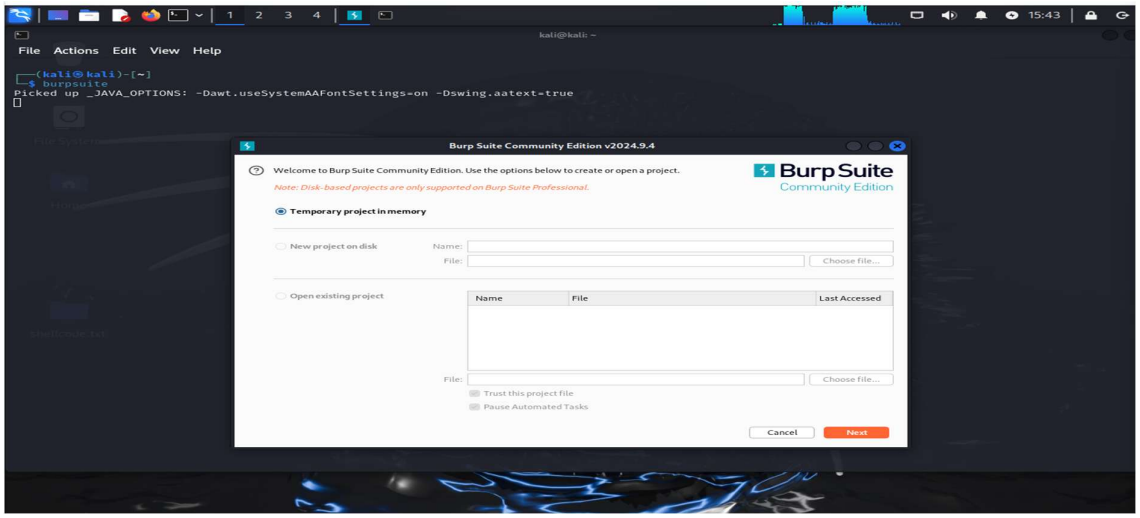
Step 3 – Create Temporary Project

When Burp Suite opened, the option **Temporary Project** was selected.

Then:

- Clicked **Next**
- Selected **Use Burp Defaults**
- Clicked **Start Burp**

This initialized Burp Suite using default settings for penetration testing.



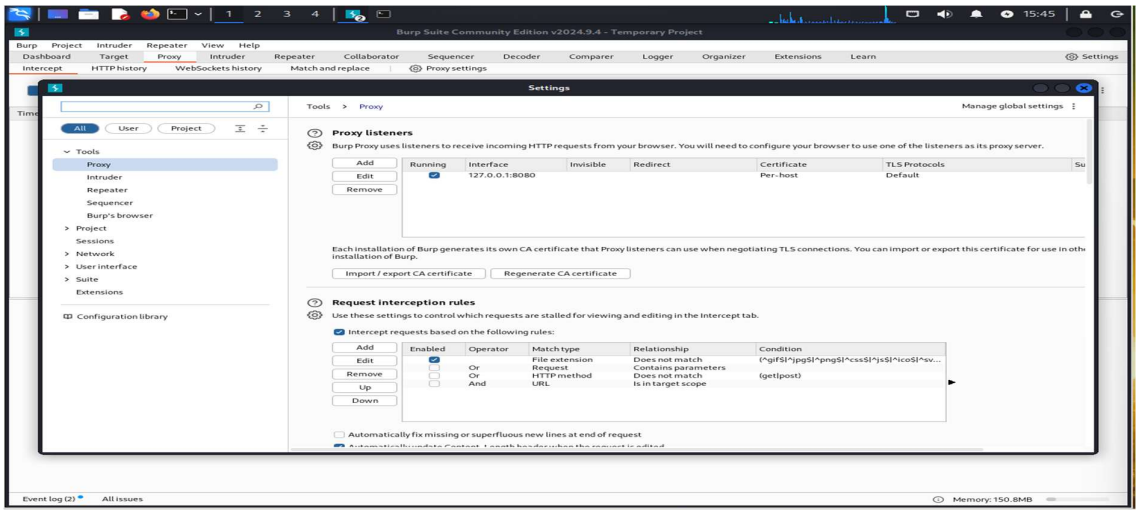
Step 4 – Verify Proxy Listener

Inside Burp Suite:

- Opened **Proxy → Proxy Settings**
- Verified that the Proxy Listener was active on:

127.0.0.1:8080

This listener is required for capturing browser traffic through Burp Suite.



Step 5 – Enable Intercept in Burp Suite

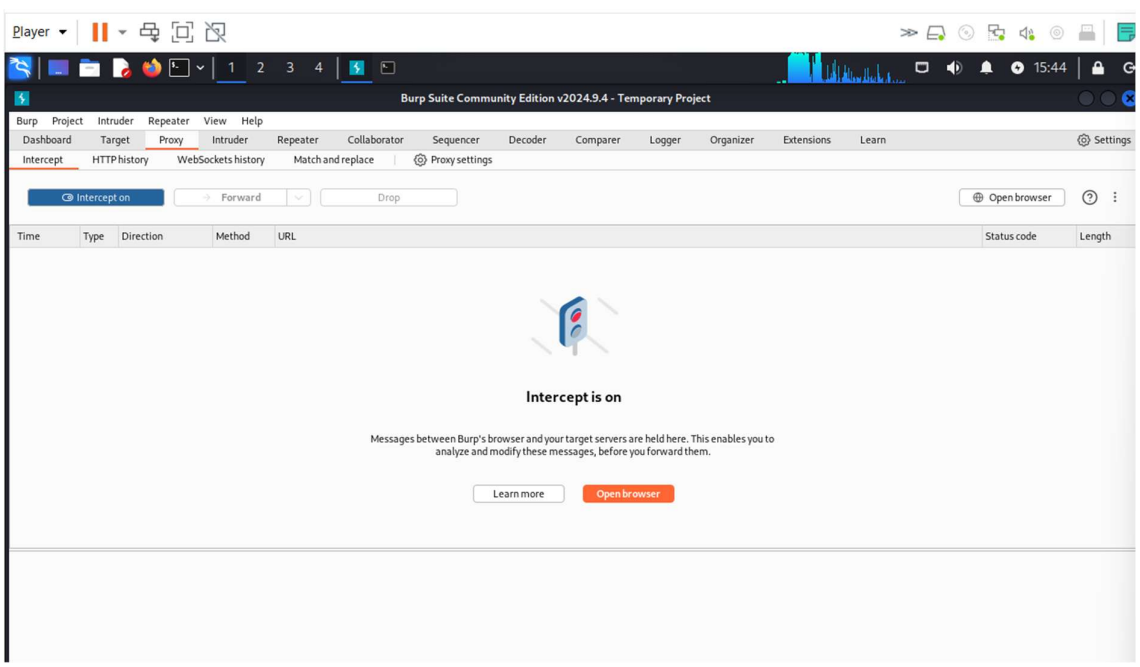
Inside Burp Suite:

- Opened **Proxy → Intercept**
- Verified that the button showed:

Intercept is ON

If it was OFF, it was clicked once to turn it ON.

This step ensured that Burp Suite could stop and capture browser requests before they reached the OWASP Juice Shop server**.**

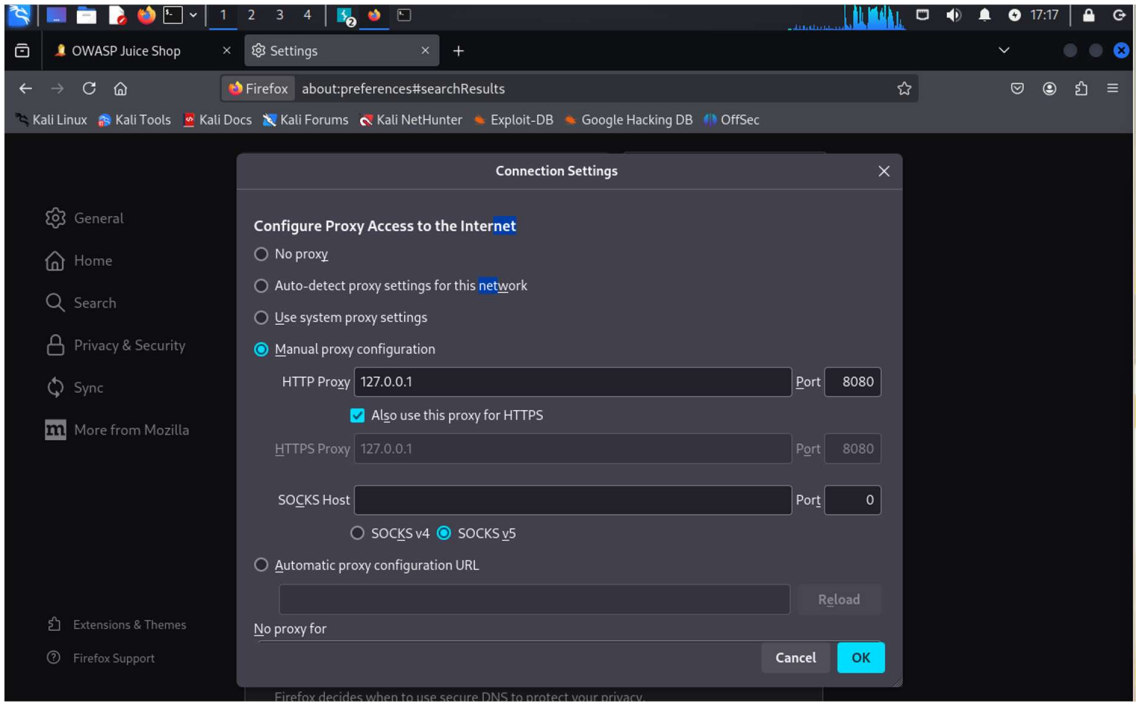


Step 6 – Configure Browser Proxy

Mozilla Firefox was opened inside Kali Linux and the proxy settings were configured manually:

- HTTP Proxy: ***127.0.0.1***
- Port: ***8080***
- Enabled: Also use this proxy for **HTTPS**

This ensured that all browser traffic passed through Burp Suite.

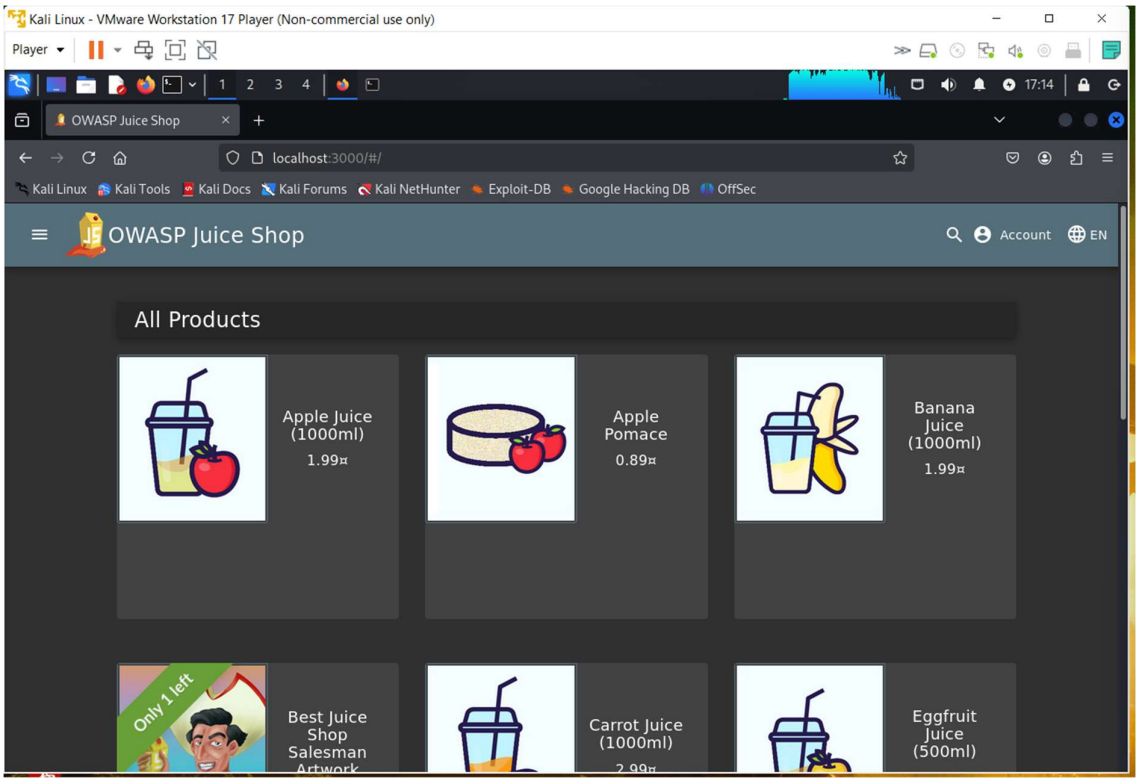


Step 7 – Open OWASP Juice Shop

OWASP Juice Shop was deployed locally using Docker and accessed using:

http://localhost:3000

The vulnerable web application loaded successfully in the browser and was used as the target for testing.



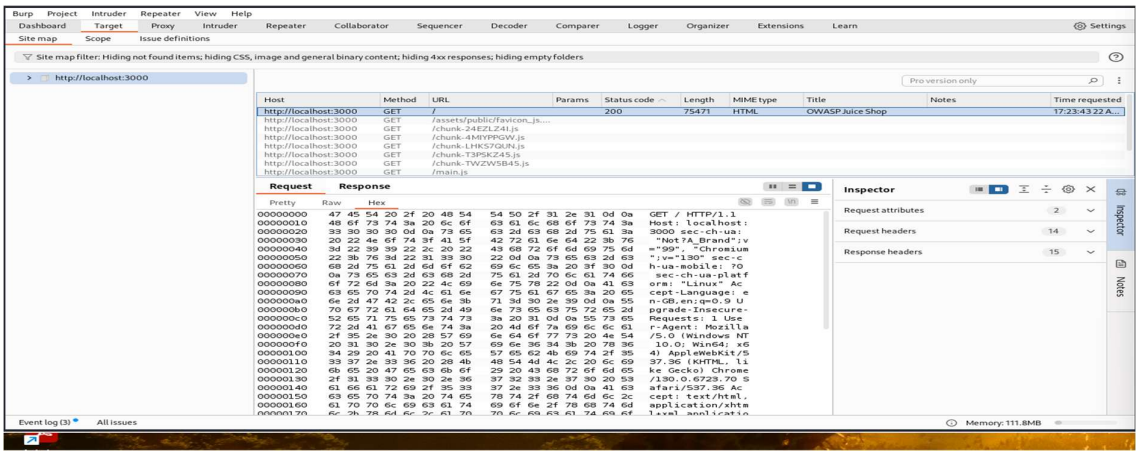
Step 8 – Verify Request Capture

After opening Juice Shop:

- Opened **Proxy → HTTP History**
- Verified that requests from:

http://localhost:3000

were being captured successfully.

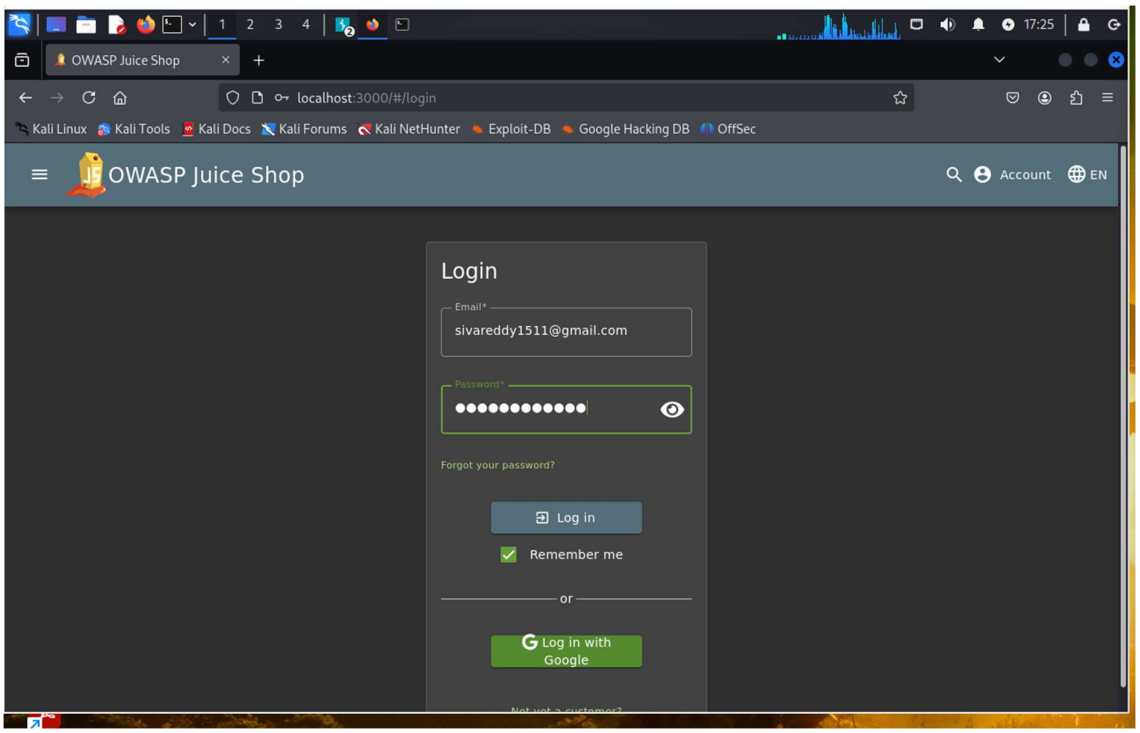


Step 9 – Open Login Page

Inside the OWASP Juice Shop application:

- Clicked on **Account**
- Opened the **Login Page**

The login functionality was selected for SQL Injection testing.



Step 10 – Capture Login Request

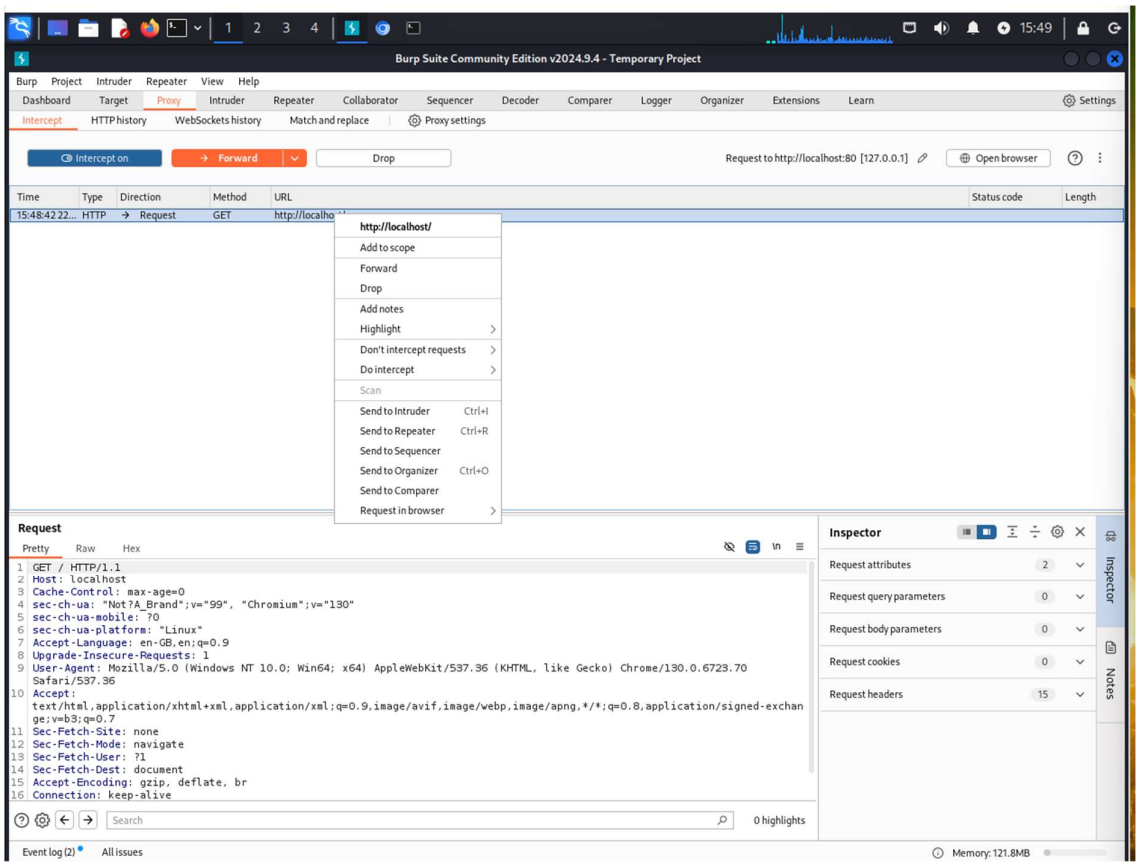
A login attempt was made using test credentials from the login page.

Burp Suite captured the request successfully.

The captured request was:

- Right-clicked
- Selected **Send to Repeater**

Burp Repeater was used to manually modify the request.



Findings Summary

ID	Vulnerability	OWASP Category	Risk Level	Status
01	SQL Injection Testing	Injection	High	Tested
02	Cross Site Scripting (XSS)	Injection	High	Tested
03	Broken Authentication	Identification and Authentication Failures	Medium	Tested
04	Security Misconfiguration	Security Misconfiguration	Medium	Observed
05	Request Interception & Response Analysis	Security Testing Process	Medium	Completed

Detailed Findings and Proof of Concept

SQL Injection Testing

Description

The login-related endpoint was manually tested for SQL Injection by modifying request parameters through Burp Repeater.

Impact

Improper input validation may allow attackers to bypass authentication or manipulate backend queries.

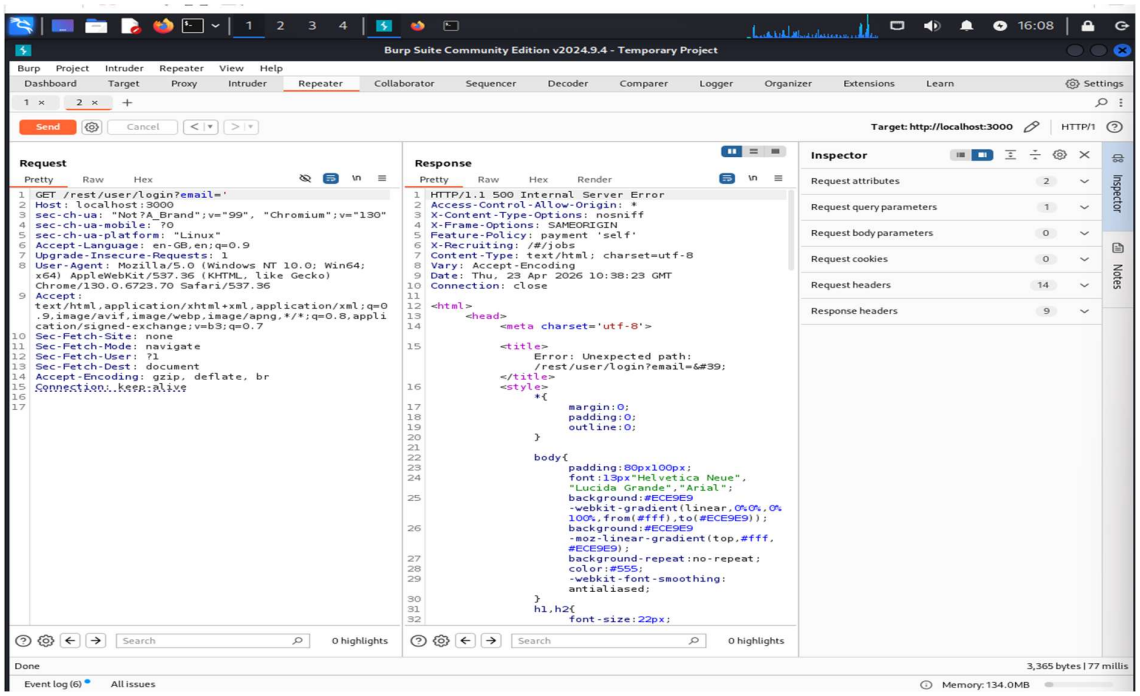
Steps Performed

1. Captured request using Burp Proxy
2. Sent request to Repeater
3. Modified endpoint: *GET /rest/user/login?email=' OR 1=1-- HTTP/1.1*
4. Sent the modified request

Result

The server returned: *HTTP/1.1 500 Internal Server Error*

This indicates improper handling of malformed input and confirms SQL Injection testing methodology



Cross Site Scripting (XSS) Testing

Description

Reflected XSS testing was performed by injecting JavaScript payloads through request parameters.

Impact

If user input is not sanitized, attackers can execute malicious JavaScript in the victim's browser.

Steps Performed

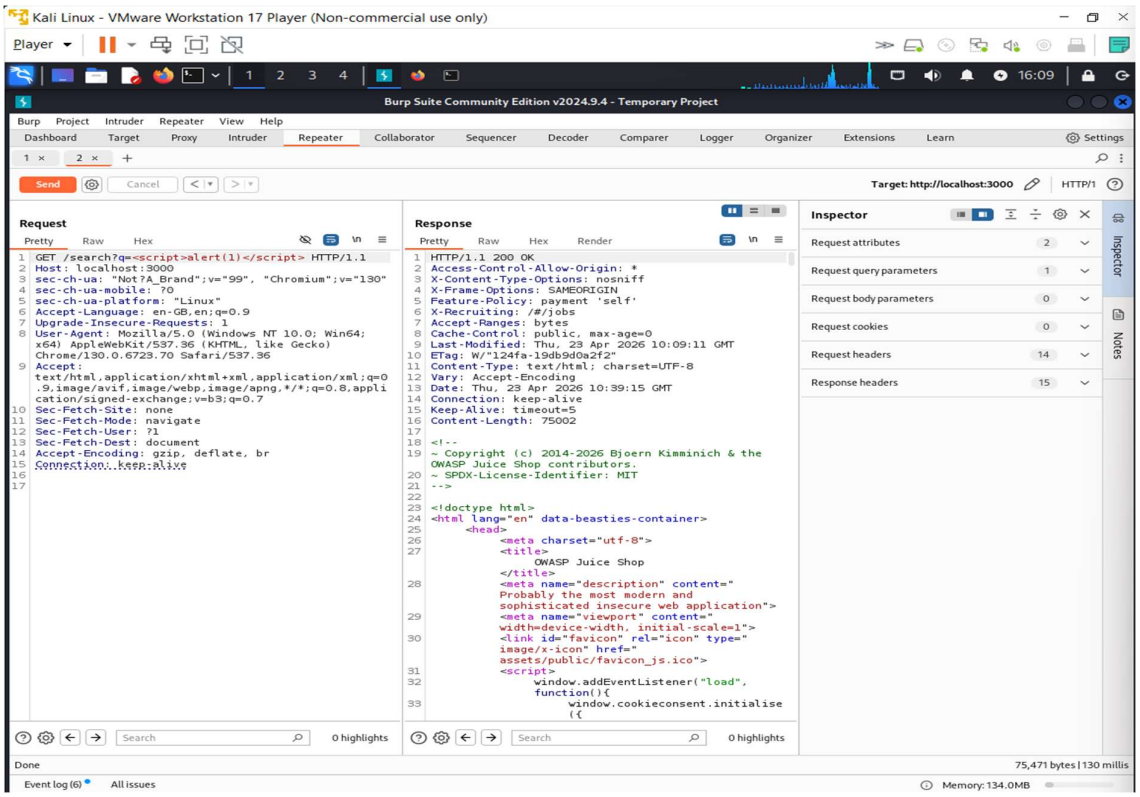
1. Sent request to Repeater
2. Modified request:

   GET /search?q=<script>alert(1)</script> HTTP/1.1

3. Sent the payload

Result

The payload was accepted and server response was captured successfully, confirming XSS testing.



Security Misconfiguration

Description

Server behaviour was analyzed using Burp Repeater and HTTP responses.

Impact

Improper error handling and exposed internal information help attackers understand backend behaviour.

Steps Performed

1. Tested endpoint:

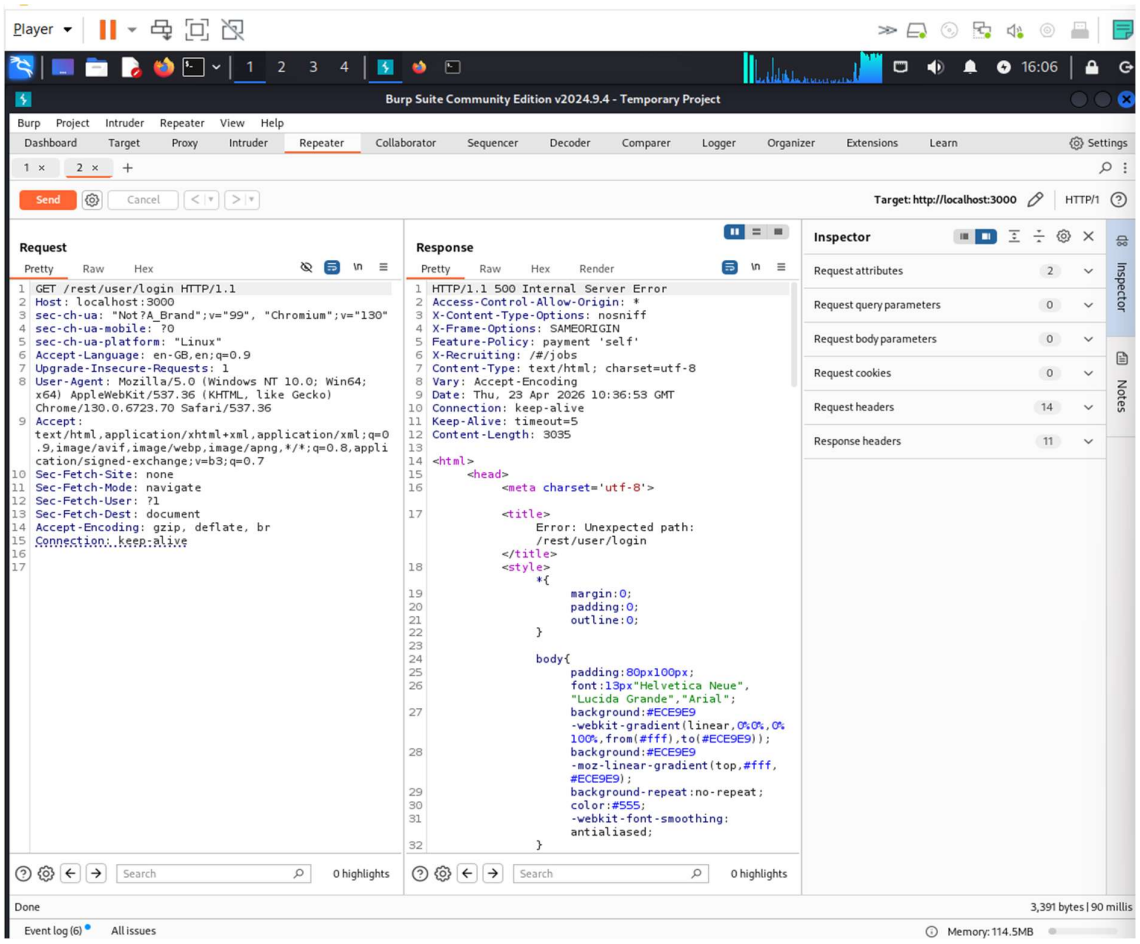
   /rest/user/login

2. Observed response:

   Unexpectedpath
   500 Internal Server Error

Result

The application exposed internal error responses, indicating security misconfiguration.



Request Interception and Response Analysis

Description

HTTP requests and responses were intercepted and analyzed using Burp Suite Proxy and HTTP History.

Impact

This helped identify application behavior, request flow, and testing opportunities.

Steps Performed

1. Verified Proxy Listener on:

   127.0.0.1:8080

2. Opened:

   http://localhost:3000

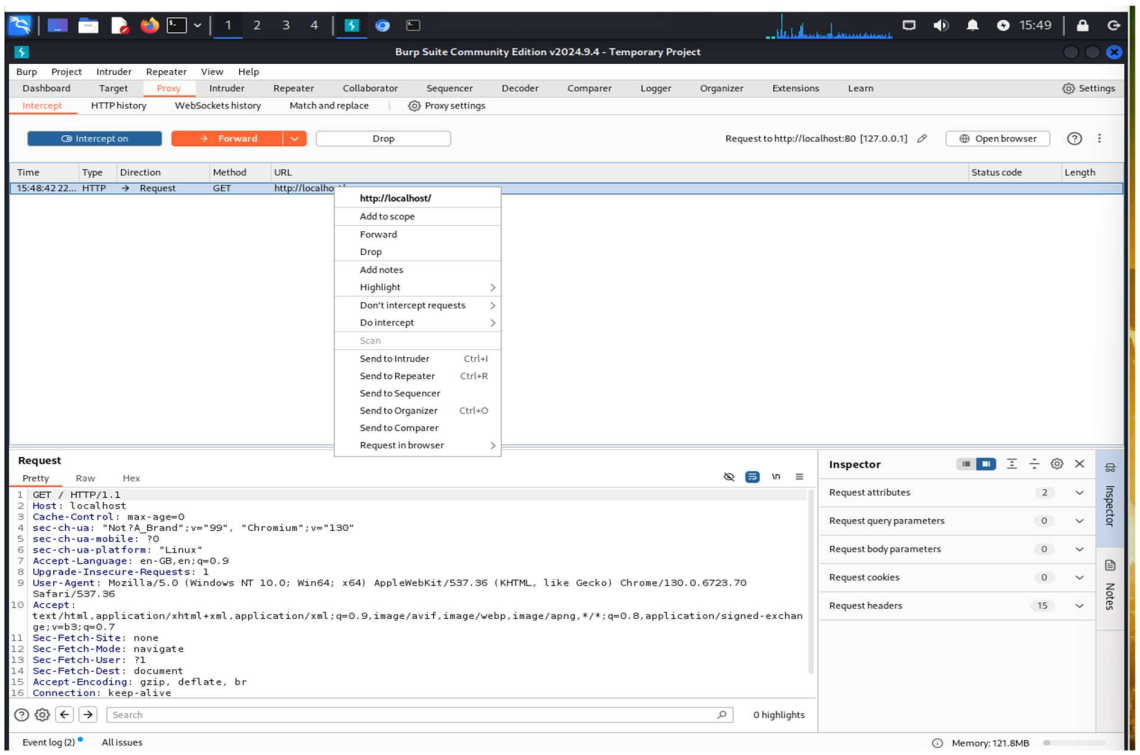
3. Captured:

   GET /

4. Sent requests to Repeater

Result

Successful request interception confirmed proper testing setup.



Broken Authentication Testing

Description

Authentication handling was tested using fake login attempts and observing system behaviour.

Impact

Weak authentication controls can allow brute-force attacks and unauthorized access attempts.

Steps Performed

1. Opened Juice Shop login page
2. Used test credentials:

Email: ***sivareddy1511@gmail.com***

Password: ***wrongpassword***

3. Observed login behaviour and server response

Result

Authentication behaviour was analyzed and request handling was observed through Burp Suite.

Remediation Recommendations

Vulnerability	Recommendation
SQL Injection	Use prepared statements and strong input validation
XSS	Sanitize input and apply output encoding
Broken Authentication	Implement rate limiting and account lockout
Security Misconfiguration	Disable verbose errors and improve exception handling
Request Security	Validate endpoints and enforce secure request handling

These measures significantly improve application security.

Conclusion

The penetration testing assessment successfully identified multiple security weaknesses in the OWASP Juice Shop web application using Burp Suite and the OWASP Top 10 methodology.

Testing included request interception, endpoint analysis, SQL Injection testing, Cross Site Scripting (XSS), authentication testing, and response analysis.

The use of Burp Suite Proxy and Repeater helped simulate attacker behavior and identify improper input handling and security misconfiguration.

Implementing secure coding practices, proper validation, and strong authentication controls will significantly improve the security posture of the application and reduce the risk of real-world attacks.